

**UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CAMPUS APUCARANA
ENGENHARIA DA COMPUTAÇÃO**

**CENÁRIOS DE TESTES E ENTREGA DO PROJETO DE GERENCIAMENTO DA
ACADEMIA LU MAFRA PERSONAL TRAINER - ENGENHARIA DE SOFTWARE**

**JOÃO ANTONIO SITTA MARTINS, MILLENA SARTORI DE OLIVEIRA, MARIA EDUARDA
MAFRA
2025**

CENÁRIOS DE TESTES DAS FUNCIONALIDADES ENTREGUES NO T2:

Validação de Campos Obrigatórios

- **Descrição do cenário:** Os campos de Nome, Data de nascimento, CPF, celular, quantidade de dias, vencimento da mensalidade e validação do termo de responsabilidade são obrigatórios. Para que o cadastro possa ser efetivado com sucesso, todos os campos previamente listados devem estar preenchidos.
- Dado que o sistema verifica se os campos obrigatórios foram preenchidos - se nenhum dos campos está vazio -, quando um campo obrigatório não é preenchido, então o cadastro é interrompido.

Exibição na Tabela de Horários

- **Descrição do cenário:** O sistema deve fornecer à cliente uma tabela contendo o número de alunas em seus respectivos horários, para que a usuária possa ter uma perspectiva geral dos cadastros realizados e as aulas do dia, além de exibir quantas vagas ainda restam.
- Dado que o sistema exibe uma tabela com os cadastros bem sucedidos, quando um cadastro mal sucedido é realizado, ele não pode estar na tabela. Por outro lado, caso o cadastro tenha sido efetuado, ele precisa constar na tabela.

Cadastro em Horário Indisponível

- **Descrição do cenário:** O sistema deve impedir que o cadastro seja realizado em um horário não disponível, seja ele fora do horário de funcionamento ou com vagas esgotadas.
- Dado que o sistema verifica se o horário está disponível, quando a cliente cadastrar uma aluna em um horário de funcionamento com vagas disponíveis, o sistema deve permitir que o cadastro siga adiante. Caso o horário escolhido esteja fora do horário comercial ou as vagas já estejam esgotadas, então o cadastro deve ser interrompido.

Verificação de CPF

- **Descrição de cenário:** O campo CPF, além de ser obrigatório, também precisa ser válido. Dessa forma, o sistema é capaz de reconhecer se o CPF ali inserido é existente ou não.
- Dado que o sistema é capaz de reconhecer se o CPF existe, se for inserido algum não existente, o cadastro deve ser interrompido.

RELATÓRIO DOS TESTES DAS FUNCIONALIDADES DO T2:

Validação de campos obrigatórios (Maria Eduarda):

Sempre ao realizar um cadastro, é preciso que todos os campos obrigatórios estejam preenchidos. Dessa forma, o sistema não permite que sejam feitos cadastros com informações obrigatórias faltantes:

```
✓ def validar_cpf(cpf: str):  
    cpf_digits = ''.join(ch for ch in cpf if ch.isdigit())  
    ✓ if len(cpf_digits) != 11:  
        |     return False  
    ✓ if cpf_digits == cpf_digits[0] * 11:  
        |     return False  
    nums = [int(c) for c in cpf_digits]  
    s = sum(nums[i] * (10 - i) for i in range(9))  
    r = s % 11  
    dv1 = 0 if r < 2 else 11 - r  
    ✓ if dv1 != nums[9]:  
        |     return False  
    s2 = sum(nums[i] * (11 - i) for i in range(10))  
    r2 = s2 % 11  
    dv2 = 0 if r2 < 2 else 11 - r2  
    ✓ if dv2 != nums[10]:  
        |     return False  
    return True
```

```
def validar_data(data: str):
    pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$'
    if not re.match(pattern, data):
        return False

    dia, mes, ano = map(int, data.split('/'))
    if mes == 2:
        if (ano % 4 == 0 and ano % 100 != 0) or (ano % 400 == 0):
            if dia > 29:
                return False
        else:
            if dia > 28:
                return False
    elif mes in {4, 6, 9, 11}:
        if dia > 30:
            return False
        return True
    else:
        if dia > 31:
            return False
        return True
```

```
DDDS_VALIDOS = [  
    '11', '12', '13', '14', '15', '16', '17', '18', '19', # SP  
    '21', '22', '24', # RJ  
    '27', '28', # ES  
    '31', '32', '33', '34', '35', '37', '38', # MG  
    '41', '42', '43', '44', '45', '46', # PR  
    '47', '48', '49', # SC  
    '51', '53', '54', '55', # RS  
    '61', # DF  
    '62', '64', # GO  
    '63', # TO  
    '65', '66', # MT  
    '67', # MS  
    '68', # AC  
    '69', # RO  
    '71', '73', '74', '75', '77', # BA  
    '79', # SE  
    '81', '87', # PE  
    '82', # AL  
    '83', # PB  
    '84', # RN  
    '85', '88', # CE  
    '86', '89', # PI  
    '91', '93', '94', # PA  
    '92', '97', # AM  
    '95', # RR  
    '96', # AP  
    '98', '99' # MA  
]
```

```
def validar_telefone(telefone: str):
    telefone = ''.join(ch for ch in telefone if ch.isdigit())
    if len(telefone) != 11:
        return False

    ddd = telefone[:2]
    if ddd not in DDDS_VALIDOS:
        return False

    padrao_celular = re.compile(r'^\d{2}9\d{8}$')
    if padrao_celular.match(telefone):
        return True
    else:
        return False
```

```

class Academia:

    def addAlunas(self, aluna: Aluna):
        aluna.diasSemana = [d.strip().lower() for d in aluna.diasSemana]
        aluna.horario = [h.strip() for h in aluna.horario]

        for dia, horario in zip(aluna.diasSemana, aluna.horario):
            with self._connect() as conn:
                cur = conn.cursor()
                # insere aluna se não existir (cpf UNIQUE evita duplicata)
                try:
                    cur.execute('''
                        INSERT INTO alunas (
                            nome, apelido, nascimento, cep, endereco, bairro,
                            celular, cpf, dias, horario, valor, vencimento, termo
                        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                    ''', (
                        aluna.nome, aluna.apelido, aluna.nascimento,
                        aluna.cep, aluna.endereco, aluna.bairro, aluna.celular,
                        aluna.cpf, aluna.dias, horario, aluna.valor,
                        aluna.vencimento, int(aluna.termo)
                    ))
                    conn.commit()
                except sqlite3.IntegrityError:
                    pass

                cur.execute('''
                    SELECT 1 FROM horarios WHERE dia = ? AND horario = ? AND aluna_cpf = ?
                ''', (dia, horario, aluna.cpf))
                if cur.fetchone():
                    return(f"{aluna.nome} já está cadastrada em {dia} às {horario}.")

                cur.execute('''
                    SELECT COUNT(*) FROM horarios WHERE dia = ? AND horario = ?
                ''', (dia, horario))
                count = cur.fetchone()[0]
                if count >= self.limiteHorario(horario):
                    return(f"O horário {horario} de {dia} já está cheio ({self.limiteHorario(horario)} alunas).")

                cur.execute('''
                    INSERT INTO horarios (dia, horario, aluna_cpf) VALUES (?, ?, ?)
                ''', (dia, horario, aluna.cpf))
                conn.commit()

        return(f"{aluna.nome} cadastrada nos horários com sucesso.")

```

LU MAFRA
Personal Trainer

Cadastro

Nome: Apellido: Data de Nascimento:

CEP: Endereço: Bairro: Celular:

CPF: Quantidade de Dias: ☐ 2x ☒ 3x Valor da Mensalidade:

Vencimento: ☐ 5 ☐ 10 ☐ 15 ☐ 20 ☒ 25 ☐ 30

Termo de Responsabilidade para Prática de Atividade Física

Estou ciente de que é recomendável conversar com um médico antes de aumentar meu nível atual de atividade física, tenho pleno conhecimento da minha atual condição de saúde. Sei também que a realização de atividades físicas pode acarretar algum risco, caso existam problemas clínicos que a contraindiquem total ou parcialmente. Assumo plena responsabilidade por qualquer atividade física praticada sem o atendimento a essa recomendação e DECLARO que aceito as responsabilidades inerentes à participação no treino, bem como isento de qualquer responsabilidade a "Lu Mafra Personal Trainer".

Concorda com o Termo de Responsabilidade para Prática de Atividade Física?

☒ Concordo

O campo CPF não foi preenchido. Portanto, o cadastro não foi efetuado e requisitou novamente a inserção do dado.

LU MAFRA
Personal Trainer

Pesquisar Alunas

Digite o nome ou CPF da aluna:

Nenhuma aluna encontrada com essa informação

Como o cadastro com informações faltantes foi interrompido, as outras informações inseridas não foram salvas no Banco de Dados.

Agora, inserindo todas as informações obrigatórias:



LU MAFRA
Personal Trainer



Cadastro

Nome:

Apelido:

Data de Nascimento:

CEP:

Buscar CEP

Endereço:

Bairro:

Celular:

CPF:

Quantidade de Dias: ☒ 2x ☐ 3x

Valor da Mensalidade:

Vencimento: ☐ 5 ☐ 10 ☐ 15 ☐ 20 ☒ 25 ☐ 30

Termo de Responsabilidade para Prática de Atividade Física

Estou ciente de que é recomendável conversar com um médico antes de aumentar meu nível atual de atividade física, tenho pleno conhecimento da minha atual condição de saúde. Sei também que a realização de atividades físicas pode acarretar algum risco, caso existam problemas clínicos que a contraindiquem total ou parcialmente. Assumo plena responsabilidade por qualquer atividade física praticada sem o atendimento a essa recomendação e DECLARO que aceito as responsabilidades inerentes à participação no treino, bem como isento de qualquer responsabilidade a "Lu Mafra Personal Trainer".

Concorda com o Termo de Responsabilidade para Prática de Atividade Física?

☒ Concordo

Continuar



LU MAFRA
Personal Trainer



Horários Disponíveis

Selecione os dias da semana e o horário:

Segunda

Segunda
Terça
Quarta
Quinta
Sexta

Selecione um t

Selecione um t

Voltar

Enviar

Como as informações são todas válidas (campos obrigatórios preenchidos), o cadastro passou para a próxima etapa. Assim, o teste de campos obrigatórios foi validado.

Exibição na Tabela de Horários (Maria Eduarda):

Sempre ao cadastrar uma aluna, ela, por consequência, terá aula em algum horário. Assim, toda aluna deve constar na tabela de horários: ela diz o número de alunas ali cadastradas e o número de vagas restantes. Por outro lado, caso o cadastro seja interrompido, a aluna não deve aparecer ali.

```
def mostraVagas(self, dia: str) -> List[Tuple[int, int, int, int, int]]:
    lista_de_vagas: List[Tuple[int, int, int, int, int]] = []

    with self._connect() as conn:
        cur = conn.cursor()
        cur.execute('SELECT DISTINCT horario FROM horarios WHERE dia = ?', (dia,))
        horarios = [row[0] for row in cur.fetchall()]

        if not horarios:
            return []
        else:
            for h in sorted(horarios):
                cur.execute('SELECT COUNT(*) FROM horarios WHERE dia = ? AND horario = ?', (dia, h))
                ocupado = cur.fetchone()[0]
                limite = self.limiteHorario(h)

                if limite == 0:
                    continue
                vagas = max(limite - ocupado, 0)
                vaga_atual = (h, ocupado, limite, vagas)
                lista_de_vagas.append(vaga_atual)

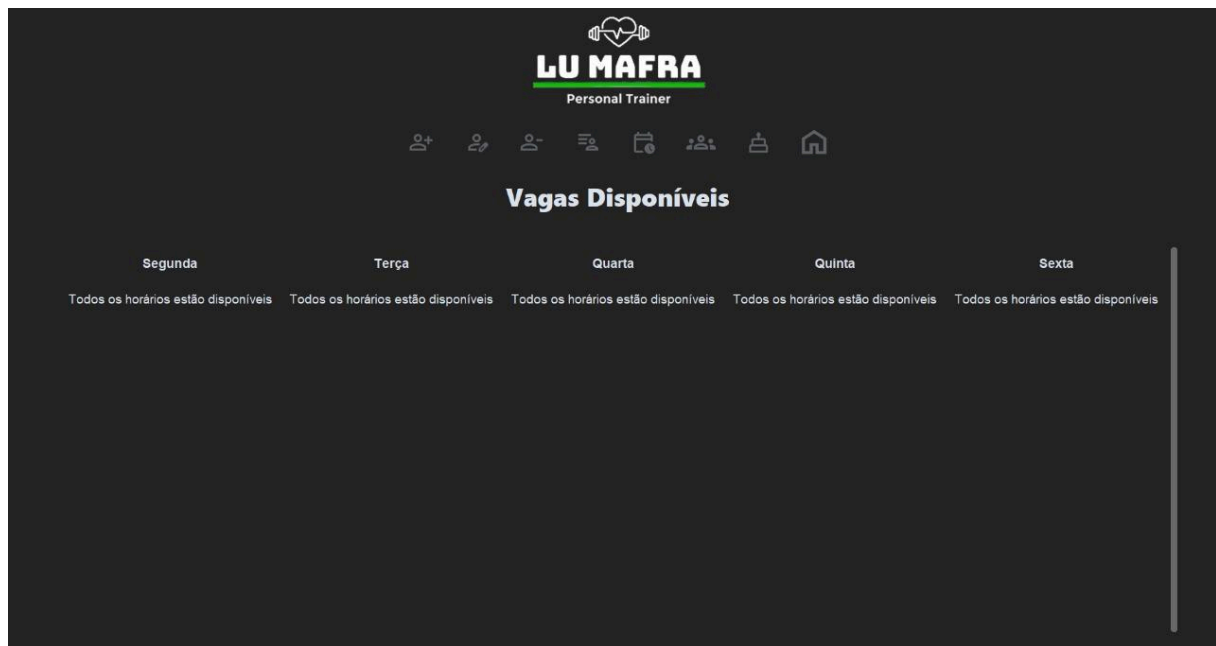
    return lista_de_vagas
```

```
def buscarPorNome_ou_Cpf(self, info_parcial: str):
    info_parcial = f"%{info_parcial.lower()}%"

    with self._connect() as conn:
        cur = conn.cursor()
        if info_parcial.replace("%", "").isdigit():
            cur.execute(""" SELECT nome, cpf, nascimento, valor, vencimento FROM alunas WHERE LOWER(cpf) LIKE ? ORDER BY nome """, (info_parcial,))
        else:
            cur.execute(""" SELECT nome, cpf, nascimento, valor, vencimento FROM alunas WHERE LOWER(nome) LIKE ? ORDER BY nome """, (info_parcial,))
        alunas = cur.fetchall()

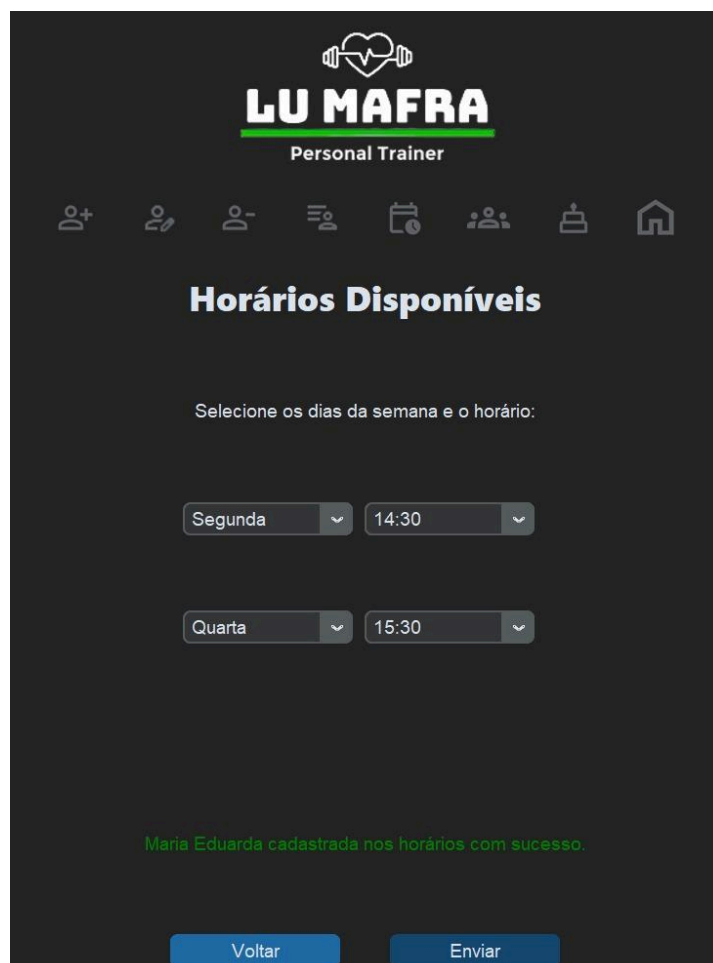
        if not alunas:
            return None

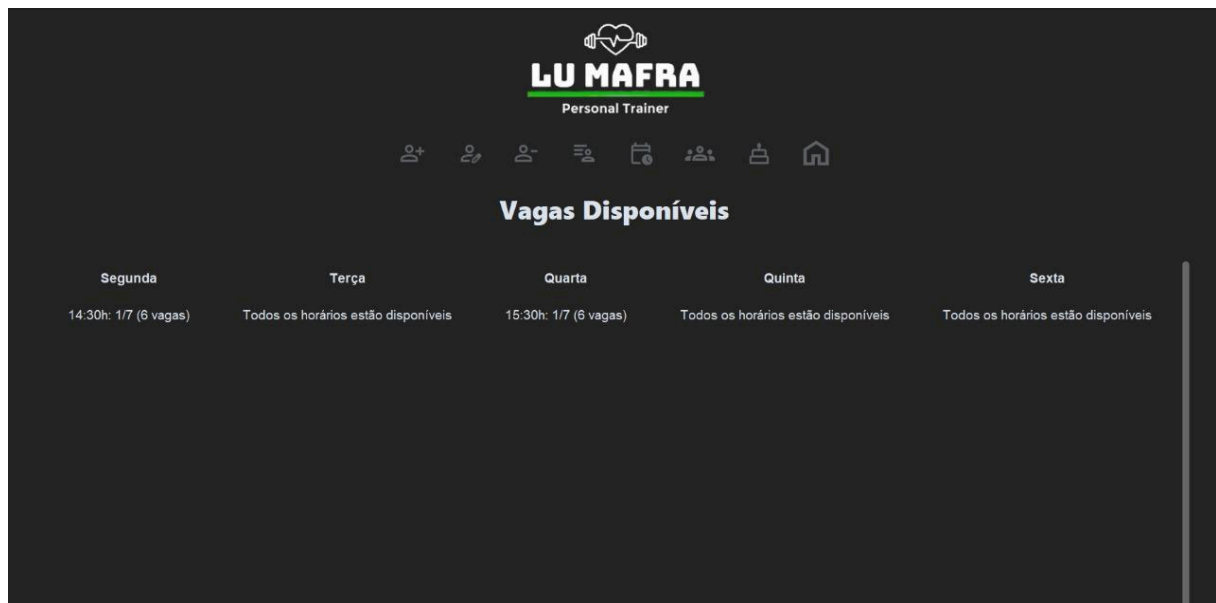
    return alunas
```



Testando com o cadastro interrompido graças à falta de informações obrigatórias, é possível notar que a tabela não recebeu nenhum dado - as informações canceladas não foram armazenadas no banco de dados.

Agora, testando com um cadastro bem sucedido:





As figuras 6 e 7 mostram que quando o cadastro é realizado com sucesso, os dados são armazenados no Banco de Dados e são inseridos corretamente na tabela. Assim, esse teste também está validado.

Cadastro em Horário Indisponível (Millena):

O sistema não deve permitir que o cadastro em horários indisponíveis seja permitido. Assim, se o horário estiver com vagas esgotadas, o cadastro é interrompido.

```
def addAlunas(self, aluna: Aluna):
    cur.execute('''
        SELECT 1 FROM horarios WHERE dia = ? AND horario = ? AND aluna_cpf = ?
    ''', (dia, horario, aluna.cpf))
    if cur.fetchone():
        return(f"{aluna.nome} já está cadastrada em {dia} às {horario}.")

    cur.execute('''
        SELECT COUNT(*) FROM horarios WHERE dia = ? AND horario = ?
    ''', (dia, horario))
    count = cur.fetchone()[0]
    if count >= self.limiteHorario(horario):
        return(f"O horário {horario} de {dia} já está cheio ({self.limiteHorario(horario)} alunas).")
```

The screenshot shows the 'Horários Disponíveis' (Available Hours) screen of the 'LU MAFRA Personal Trainer' app. At the top, there is a logo with a heart and dumbbells, followed by the text 'LU MAFRA' and 'Personal Trainer'. Below this is a navigation bar with icons for adding a user, editing a user, deleting a user, a list, a calendar, a group of people, a birthday cake, and a home icon. The main heading is 'Horários Disponíveis'. Below it, a prompt says 'Selecione os dias da semana e o horário:'. There are two rows of dropdown menus. The first row shows 'Segunda' (Monday) and '14:30'. The second row shows 'Quarta' (Wednesday) and '19:30'. A green message at the bottom states 'O horário 14:30 de segunda já está cheio (7 alunas)'. At the very bottom, there are two buttons: 'Voltar' (Back) and 'Enviar' (Send).

LU MAFRA
Personal Trainer

Horários Disponíveis

Selecione os dias da semana e o horário:

Segunda 14:30

Quarta 19:30

O horário 14:30 de segunda já está cheio (7 alunas).

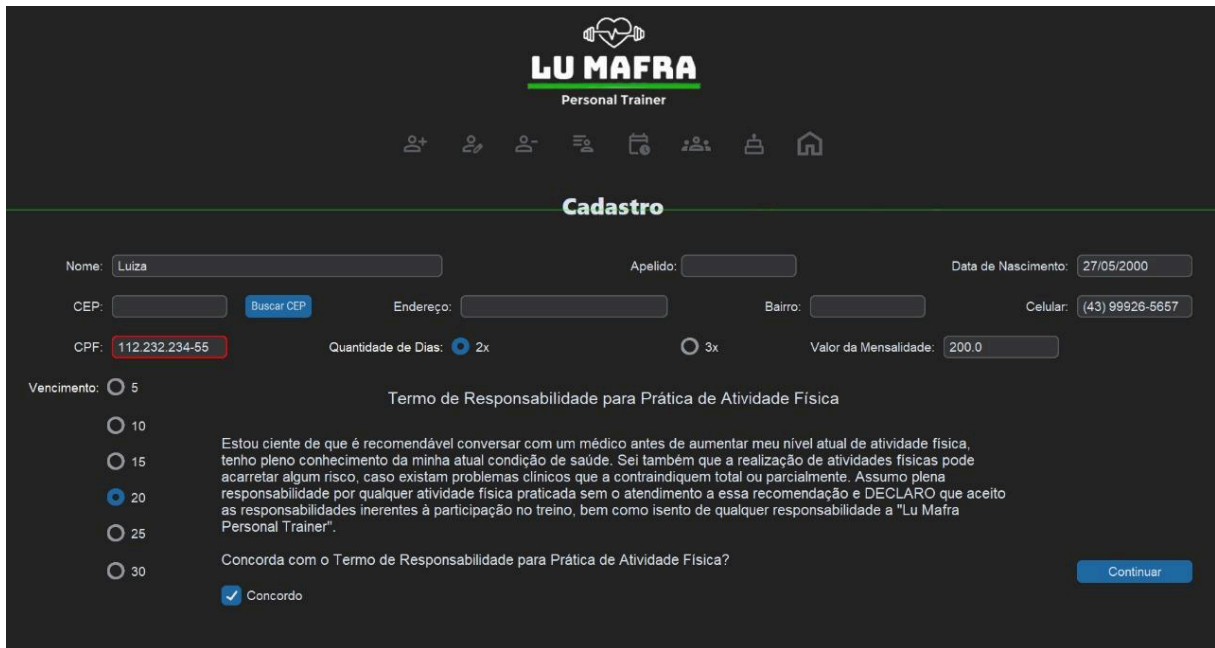
Voltar Enviar

Na figura 8 é exibido o aviso de que as vagas já estão indisponíveis para o horário selecionado. Assim, o teste está validado.

Verificação de CPF (Millena):

Para testar a validação do CPF, é necessário inserir algum que não exista. Em seguida, é esperado que o cadastro seja interrompido!

```
def validar_cpf(cpf: str):
    cpf_digits = ''.join(ch for ch in cpf if ch.isdigit())
    if len(cpf_digits) != 11:
        return False
    if cpf_digits == cpf_digits[0] * 11:
        return False
    nums = [int(c) for c in cpf_digits]
    s = sum(nums[i] * (10 - i) for i in range(9))
    r = s % 11
    dv1 = 0 if r < 2 else 11 - r
    if dv1 != nums[9]:
        return False
    s2 = sum(nums[i] * (11 - i) for i in range(10))
    r2 = s2 % 11
    dv2 = 0 if r2 < 2 else 11 - r2
    if dv2 != nums[10]:
        return False
    return True
```



LU MAFRA
Personal Trainer

Cadastro

Nome: Apellido: Data de Nascimento:

CEP: Endereço: Bairro: Celular:

CPF: Quantidade de Dias: ☒ 2x ☐ 3x Valor da Mensalidade:

Vencimento: ☐ 5 ☐ 10 ☐ 15 ☒ 20 ☐ 25 ☐ 30

Termo de Responsabilidade para Prática de Atividade Física

Estou ciente de que é recomendável conversar com um médico antes de aumentar meu nível atual de atividade física, tenho pleno conhecimento da minha atual condição de saúde. Sei também que a realização de atividades físicas pode acarretar algum risco, caso existam problemas clínicos que a contraindiquem total ou parcialmente. Assumo plena responsabilidade por qualquer atividade física praticada sem o atendimento a essa recomendação e DECLARO que aceito as responsabilidades inerentes à participação no treino, bem como isento de qualquer responsabilidade a "Lu Mafra Personal Trainer".

Concorda com o Termo de Responsabilidade para Prática de Atividade Física?

☒ Concordo

Como o teste comprova a validade da funcionalidade, o teste foi validado.

NOVAS FUNCIONALIDADES

Verificação de Login

- **Descrição do cenário:** O sistema faz verificação de login. A motivação é por questões de segurança, dado que o sistema deve ser permitido apenas aos funcionários do estabelecimento.

- Dado que o sistema verifica se o usuário e senha estão cadastrados no sistema, o login só deve ser permitido àqueles que inserirem o nome de usuário e senha corretos. Caso contrário, o acesso deve ser negado.

Lembrete de Aniversário

- **Descrição do cenário:** Por meio de pesquisa, a cliente deve ser capaz de selecionar um mês em específico e ser informada de quais alunas são aniversariantes.
- Dado que o sistema armazena em seu banco de dados a data de nascimento de todas as alunas, todas as alunas devem aparecer nas pesquisas correspondentes aos seus meses de aniversário. Dessa forma, uma aluna não deve aparecer na pesquisa para um mês que não corresponda ao de seu aniversário.

Pesquisar Aluna

- **Descrição de cenário:** A cliente deve ser capaz de pesquisar uma aluna por meio de seu nome ou CPF.
- Caso o CPF ou o nome não existam, a pesquisa não deve resultar em nada. No entanto, caso os nomes/CPF estejam armazenados no BD, a consulta deve exibi-los.

Editar Aluna

- **Descrição de cenário:** A cliente deve ser capaz de alterar as informações inseridas a qualquer instante. Ela informará ao sistema o CPF e, assim, insere quais informações serão alteradas.
- Dado que o sistema deve armazenar as novas informações inseridas, sempre que for realizada uma pesquisa, as alterações devem permanecer e serem exibidas.

Excluir Aluna

- **Descrição do cenário:** A cliente deve ser capaz de excluir qualquer aluna a qualquer instante.
- Dado que a exclusão foi feita com sucesso, a aluna não deverá mais constar no banco de dados do sistema. Assim, ao realizar uma pesquisa, ela não deverá mais aparecer.

Cadastro de Usuário para utilização e segurança do sistema

- **Descrição do cenário:** O sistema deve usar a tabela de usuário para a criação de um local para inicializar e acessar o programa pedindo nome e senha do usuário, além disso, foi feito um leve sistema de segurança em cima disso pelo programa para não permitir que a senha seja facilmente manipulada por outros.

- Dado que o sistema verifica caso o usuário já exista na hora de registrar ou a senha seja feita de maneira inconsistente, o sistema não cria a conta ou não faz login no usuário.

RELATÓRIO DOS TESTES DAS NOVAS FUNCIONALIDADES IMPLEMENTADAS: Pesquisar aluna (João):

Para testar se a funcionalidade de pesquisa está funcionando, foi pesquisado tanto alguém que está no Banco de Dados como também alguém que não esteja.



The screenshot shows the LU MAFRA Personal Trainer app interface. At the top, there's a logo with a heart and hands, followed by "LU MAFRA" and "Personal Trainer". Below this is a navigation bar with icons for adding, editing, deleting, and viewing students, as well as a home icon. The main section is titled "Pesquisar Alunas". It features a search input field with the text "Maria Eduarda" and a blue "Pesquisar" button. Below the search bar, a table displays the search results for the entered name.

Nome	CPF	Data de Nascimento	Valor	Vencimento
Maria Eduarda	11734664959	21/10/2005	R\$200.00	25

Aqui, a pesquisa de uma aluna já cadastrada foi bem sucedida: as informações mais importantes foram exibidas conforme a cliente havia pedido.



The screenshot shows the same LU MAFRA Personal Trainer app interface. The search input field now contains the text "Juliana". The "Pesquisar" button is still present. Below the search bar, the message "Nenhuma aluna encontrada com essa informação" is displayed, indicating that no results were found for the entered name.

Na figura anterior, foi realizada a pesquisa de alguém cujo nome (ou CPF) não constasse nos cadastros realizados. Como esperado, a pesquisa não encontrou ninguém com as informações correspondentes e informou a respeito. Assim, o teste foi validado.

Lembrete de Aniversário (João):

Como a cliente pediu que houvesse uma funcionalidade voltada para ser lembrada dos aniversários das alunas, foi desenvolvido um no qual, por meio da pesquisa de um mês, tem-se uma listagem das alunas aniversariantes.

```
        return alunas

def aniversariantesMes(self, mes: int):
    mes_str = f"{mes:02d}" #eh pra 01, 02 etc ser valido tb

    with self._connect() as conn:
        cur = conn.cursor()
        cur.execute(""" SELECT nome, nascimento FROM alunas WHERE substr(nascimento, 4, 2) = ? ORDER BY nascimento """, (mes_str,))
        rows = cur.fetchall()

        if not rows:
            return None

        return rows

def validar_cpf(cpf: str):
    cpf_digits = ''.join(ch for ch in cpf if ch.isdigit())
    if len(cpf_digits) != 11:
        return False
```


LU MAFRA
Personal Trainer



Aniversariantes do Mês

Qual mês deseja verificar:

Verificar

Nome	Data de Nascimento
Maria Eduarda	21/10/2005



Aqui, vê-se que a listagem funciona corretamente: ela exibe alunas cadastradas com aniversário correspondente ao mês selecionado, além de também mostrar uma lista vazia caso não haja nenhuma.

Cadastro de usuário e verificação de login (João e Maria Eduarda):

Para o teste de cadastro de usuário, foi necessário cadastrar um nome e senha, antes de tudo. Depois, foi testado o login utilizando os dados previamente utilizados e inserções incorretas.

```

class Usuario:
    """
    Classe simples para representar usuário com nome e senha (armazenada como hash com salt).
    """
    def __init__(self, nome: str, pwd_hash: str = None, salt: str = None):
        self.nome = (nome or "").strip()
        self.pwd_hash = pwd_hash # hex string
        self.salt = salt # hex string

    def set_password(self, senha: str):
        """Gera salt curto e armazena o hash SHA256(salt + senha)."""
        if senha is None:
            raise ValueError("Senha não pode ser None")
        # salt simples suficiente para uso local; para produção use PBKDF2/etc.
        self.salt = secrets.token_hex(8) # 8 bytes -> 16 hex chars
        self.pwd_hash = hashlib.sha256((self.salt + senha).encode('utf-8')).hexdigest()

    def check_user(self, nome: str) -> bool:
        """Verifica se o user bate com o user armazenado."""
        if not (self.nome and self.pwd_hash and self.salt):
            return False
        return self.nome

    def check_password(self, senha: str) -> bool:
        """Verifica se a senha bate com o hash armazenado."""
        if not (self.pwd_hash and self.salt):
            return None
        tentativa = hashlib.sha256((self.salt + (senha or "")).encode('utf-8')).hexdigest()
        return self.pwd_hash == tentativa

    def to_db_tuple(self) -> tuple:
        """(nome, pwd_hash, salt) - para INSERT no DB."""
        return (self.nome, self.pwd_hash, self.salt)

    @classmethod
    def from_db_row(cls, row: tuple):
        """Cria Usuário a partir de linha (id, nome, pwd_hash, salt) ou (nome,pwd_hash,salt)."""
        if not row:
            return None
        if len(row) == 4:
            _, nome, pwd_hash, salt = row
        elif len(row) == 3:
            nome, pwd_hash, salt = row
        else:
            raise ValueError("Formato de row inválido")
        return cls(nome=nome, pwd_hash=pwd_hash, salt=salt)

```



Cadastro de Usuário

Nome de Usuário: Senha:

Usuário cadastrado com sucesso!



Lu Mafra Personal Trainer

Usuário:

Senha:



Login



Novo Usuário

Usuário ou senha incorretos.



Lu Mafra Personal Trainer

Usuário:

Senha:

 Login

 Novo Usuário



Conforme as figuras 15 e 16 indicam, o login foi um sucesso: para acessar o sistema, é necessário ter usuário e senha previamente cadastrados, proporcionando maior segurança ao cliente. O teste foi validado.

Excluir aluna (Millena):

Para que a aluna possa ser excluída, é necessário fazer uma busca antes. Assim, para confirmar que a aluna de fato foi excluída nos testes, foi feita uma segunda pesquisa com o mesmo nome a fim de constar se o nome foi realmente excluído.

```

283
284 def excluirAluna(self, cpf: str):
285     cpf_limpo = ''.join(filter(str.isdigit, cpf))
286
287     if len(cpf_limpo) != 11:
288         return "CPF inválido."
289
290     with self._connect as conn:
291         cur = conn.cursor()
292         #verificando se a aluna existe
293         cur.execute("SELECT nome FROM alunas WHERE cpf = ?", (cpf_limpo,))
294         row = cur.fetchone()
295
296         if not row:
297             return "Aluna não encontrada."
298
299         nome = row[0]
300         #excluindo
301         cur.execute("DELETE FROM alunas WHERE cpf = ?", (cpf_limpo,))
302         conn.commit()
303
304     return f"Aluna {nome} removida com sucesso."
305

```



LU MAFRA

Personal Trainer

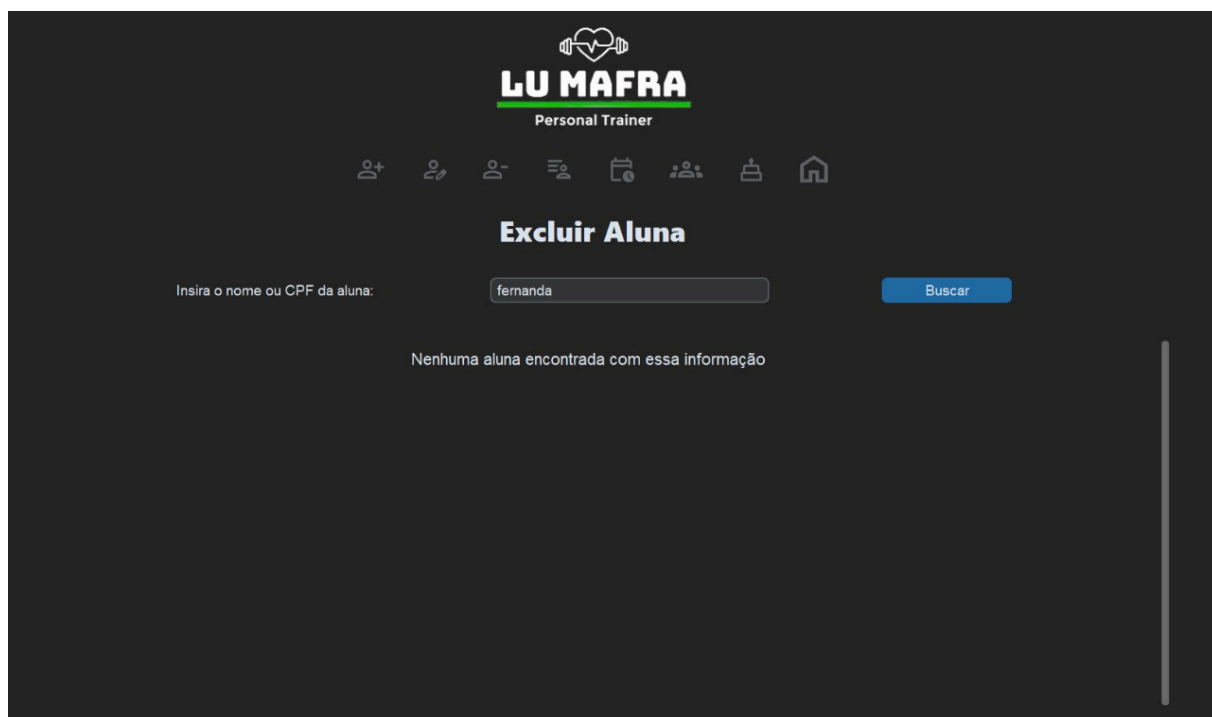
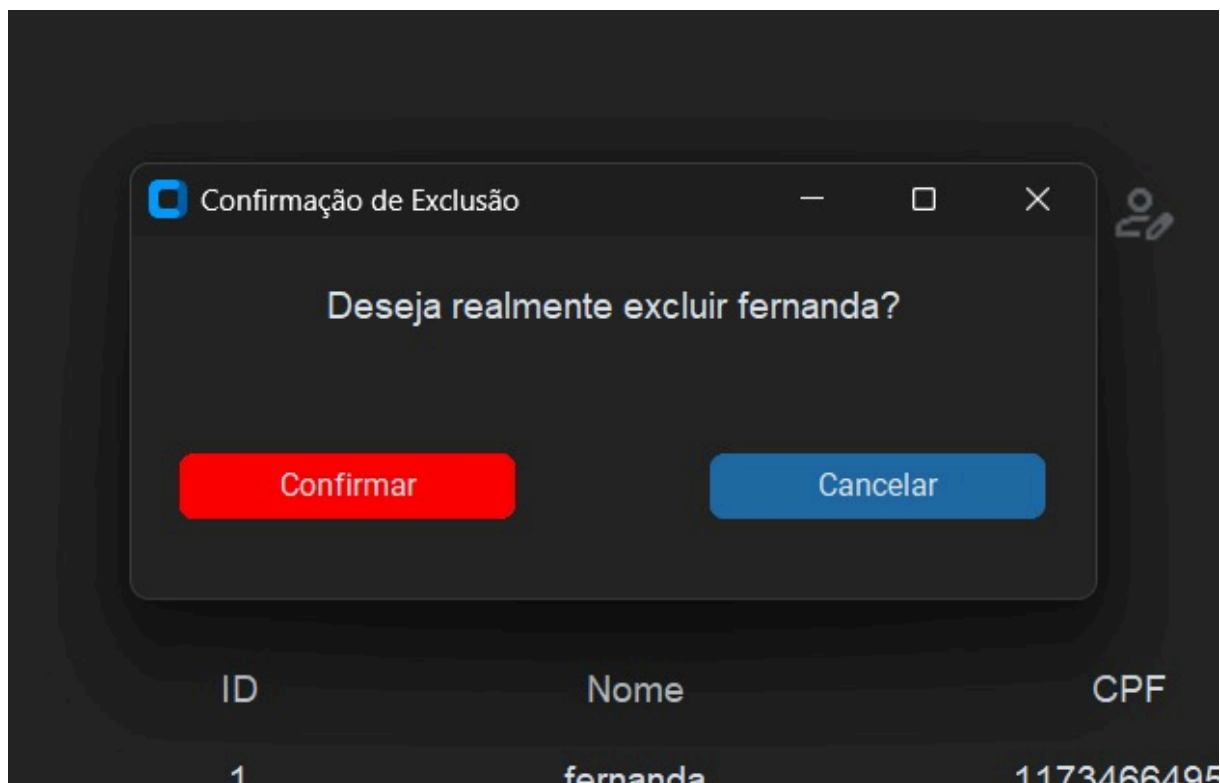
+ o - ≡ 📅 👤 🏠

Excluir Aluna

Insira o nome ou CPF da aluna:

Buscar

ID	Nome	CPF	Valor	Vencimento	ID	Excluir
1	fernanda	11734664959	R\$200.00	25		



Ao pesquisar novamente a aluna “Fernanda”, seu nome não consta mais nos resultados. Isso confirma a sua exclusão. Dessa forma, o teste foi validado.

Editar aluna (Maria Eduarda):

```

def editarAluna(self, cpf: str, **novos_dados):
    cpf_limpo = ''.join(filter(str.isdigit, cpf))

    with self._connect() as conn:
        cur = conn.cursor()

        cur.execute("SELECT * FROM alunas WHERE cpf = ?", (cpf_limpo,))
        row = cur.fetchone()

        if not row:
            return "Aluna não encontrada."

        #atualizando a tabela de alunas
        campos_validos = {"nome", "apelido", "nascimento", "cep", "endereco", "bairro",
                           "celular", "dias", "valor", "vencimento", "termo"}

        updates = []
        valores = []

        for campo, valor in novos_dados.items():
            if campo in campos_validos:
                updates.append(f"{campo} = ?")
                valores.append(valor)

        if updates:
            sql = f"UPDATE alunas SET {' '.join(updates)} WHERE cpf = ?"
            valores.append(cpf_limpo)
            cur.execute(sql, valores)
            conn.commit()

        #atualizando os dias e horarios
        diasSemana = novos_dados.get("diasSemana")
        horarios = novos_dados.get("horario")

```

```

        if diasSemana and horarios:
            diasSemana = [d.strip().lower() for d in diasSemana]
            horarios = [h.strip() for h in horarios]

            if len(diasSemana) != len(horarios):
                return "Quantidade de dias e horarios nao coincide."

            cur.execute("SELECT dia, horario FROM horarios WHERE aluna_cpf = ?", (cpf_limpo,))
            horarios_antigos = cur.fetchall()

            #apagando os horarios atuais
            cur.execute("DELETE FROM horarios WHERE aluna_cpf = ?", (cpf_limpo,))
            conn.commit()

            for d, h in zip(diasSemana, horarios):
                cur.execute("SELECT COUNT(*) FROM horarios WHERE dia = ? AND horario = ?", (d, h))
                ocupados = cur.fetchone()[0]

                limite = self.limiteHorario(h)

                if ocupados >= limite:
                    #volta os horarios antigos se der ruim
                    for da, ho in horarios_antigos:
                        cur.execute("INSERT INTO horarios (dia, horario, aluna_cpf) VALUES (?, ?, ?)", (da, ho, cpf_limpo))
                        conn.commit()
                    return f"O horário {h} de {d} ja esta cheio ({limite} alunas)."

            for d, h in zip(diasSemana, horarios):
                cur.execute("INSERT INTO horarios (dia, horario, aluna_cpf) VALUES (?, ?, ?)", (d, h, cpf_limpo))
                conn.commit()

    return "Dados da aluna atualizados com sucesso."

```